Rust Intro

Administrivia

- Anonymous Questions on Ed
- HW3 Progress?
- HW4 Releases Today (Rust Intro)
- Feedback Form
- Quiz

Agenda

- Let’s make something in Rust!
- Rust principles
- More demos
- More Rust ideals
- …

Last Time: Rust Intro

- A language with more rules to give us more safety
  - Undefined behavior is impossible in Rust
- Data-race freedom (no shared mutable references (without synchronization))
- Ownership (compiler-inserted free())
- Better types & Lifetimes (can only return provably valid references)

Hello World in Rust

```rust
fn main() {
    println!("Hello, World!");
}
```

```rust
fn main() {
    let unit = "CSE";
    let course_num: u16 = 334;
    let term = String::from("Winter 2026");

    println!("Hello {} {}, {} edition", unit, course_num, term);
}
```

hello_cse334.rs

```
$ rustc hello_cse334.rs
$ ./hello_cse334
Hello CSE 334, Winter 2026 edition
```

Clarification: Can we return heap pointers still?

In Rust, data existing on the heap vs stack is *part of its type*

- References can exist to any sort of variable

Learning Barrier!

- 1 question on Rust’s Theory

Demo: First Rust Program

- cargo
- functions
- arbitrary scopes
- summing a vec
- calling the sum function twice

Quiz on Gradescope (10:30-10:36)

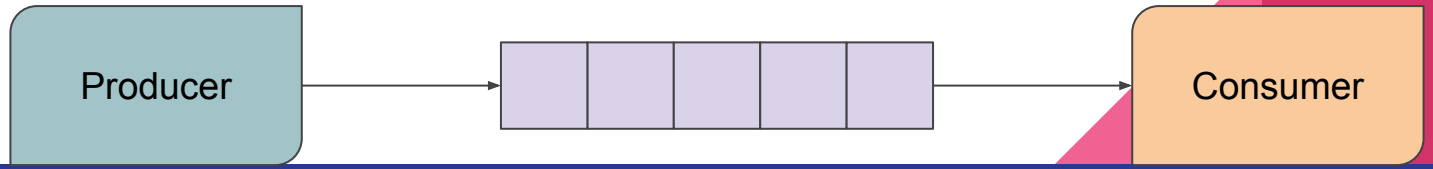
Please sit 1 seat apart from your neighbor!

Quiz on Gradescope (10:30-10:36)

Q1 Code:

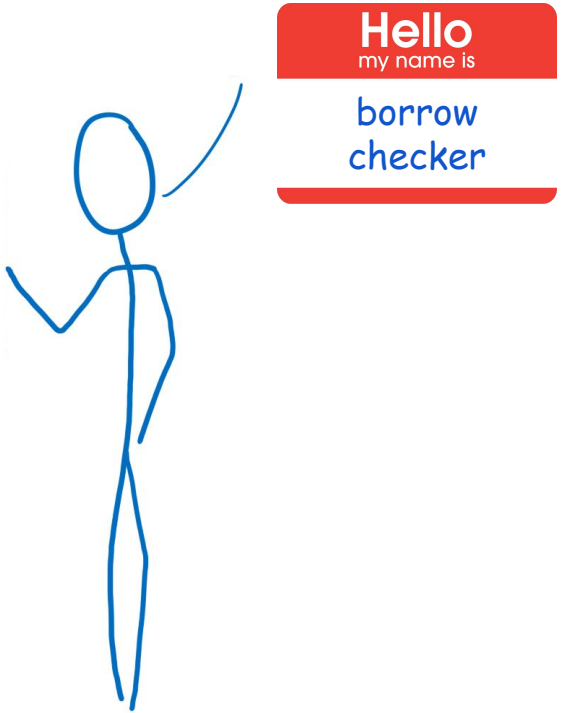
```
fn say_hello(v: Vec<i32>) -> i32 {...}

// in main, with v defined as a Vec<i32>
say_hello(v);
let v2 = v;
```

Q2: You are implementing a bounded buffer (producer writes, consumer reads). Say you are using a *single* **Semaphore(4)** to protect the internal circular data buffer, which has 5 entries, from too many writes causing lost data. What behavior will you see?



Ownership

- Ownership Rules:
  - Each value in Rust has an owner
  - There can only be one owner at a time
  - When the owner goes out of scope, the value is dropped



Ownership

- In Rust, `let a = b;` means transferring ownership from b to a
  - So does a function call f(b)
- `b` is completely unusable after this line.
- However, if the type of `b` “is `Copy`”, then `b` is still usable after the `=`
  - This is true for most primitives (i32)
  - `Copy` means that a simple bit-wise memcpy duplicates a type’s value correctly

Borrowing

- References “borrow” a value, but never take ownership
- Can have **shared references** (&T),

  or **mutable references** (&mut T)

- One mutable reference, or multiple shared references, but never both!
- Conceptually somewhere between a C pointer and a C++ reference
- Use * to dereference and copy the underlying value

Scalar Types

- signed integers: `i8`, `i16`, `i32`, `i64`, `i128` and `isize` (pointer size)
- unsigned integers: `u8`, `u16`, `u32`, `u64`, `u128` and `usize` (pointer size)
- floating point: `f32`, `f64`
- `char` Unicode scalar values like `'a'`, `'α'` and `'∞'` (4 bytes each)
- `bool` either `true` or `false`

Scalar Types

```rust
fn main() {
    // Variables can be type annotated.
    let logical: bool = true;

    let a_float: f64 = 1.0; // Regular annotation
    let an_integer = 5i32; // Suffix annotation

    // A type can also be inferred from context
    let mut inferred_type = 334; // Type i64 is inferred from another line
    inferred_type = 3344444444i64;
}
```

Special Types

- Functions have a type `fn`
- The unit type `()`, whose only possible value is an empty tuple: `()`
  - functions that don't return a type implicitly return a `()`
- and the never type `!`, which is impossible to construct ❗

Special Types

```rust
fn main() -> ! {
    print!("for");
    loop {
        print!("ever and ");
    }
}
```

Compound Types

- arrays like `[1, 2, 3; 3]`
- tuples like `(1, true, 2, false)`

User-Defined Types

- Structs (2 flavors)
- Enums
  - Combination of C-enum and C-union

Learning Barrier!

- 1 question on types

Demo

- Factorial
- Factorial with vector
- Largest item in array
- 3D Vector (structs)

Bonus Reading

https://github.com/rust-lang/rust/pull/144232